

**Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera**

LAPORAN TUGAS BESAR

IF25-21013 Strategi Algoritma

Semester Genap Tahun 2026/2027

Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Robocode Tank Royale



Disusun oleh:

Andreas Pratama Silalahi - 124140148

Feldy - 124140083

Jojo Marpaung - 124140191

Nama Kelompok: **BotAbis**

2026

DAFTAR ISI

DAFTAR ISI	2
DAFTAR TABEL	3
BAB 1	4
1. DESKRIPSI TUGAS	4
1.1. Spesifikasi Wajib	4
1.2. Komponen Skor	4
BAB 2	5
2. LANDASAN TEORI	5
2.1. Algoritma Greedy	5
2.2. Cara Kerja Program	5
BAB 3	8
3. APLIKASI STRATEGI GREEDY	8
3.1. Mapping Persoalan ke Elemen Greedy	8
3.2. Eksplorasi Alternatif Solusi Greedy	8
3.3. Analisis Efisiensi dan Efektivitas	9
Tabel 1. Analisis Efisiensi dan Efektivitas Bot	9
3.4. Strategi Greedy yang Dipilih	10
BAB 4	12
4. IMPLEMENTASI DAN PENGUJIAN	12
4.1. implementasi (Pseudocode)	12
4.1.1. BotPredator	12
4.1.2. BotSweeper	13
4.1.3. BotSentinel	13
4.1.4. BotSniper	14
4.2. Struktur Data, Fungsi, dan Prosedur	15
4.3. Pengujian	15
4.4. Analisis Hasil Pengujian	15
BAB 5	17
5. KESIMPULAN DAN SARAN	17
5.1. Kesimpulan	17
5.2. Saran	17
LAMPIRAN	18
Checklist Spesifikasi	18
Tautan Repository GitHub	18
Tautan Video (Bonus)	18
DAFTAR PUSTAKA	19

DAFTAR TABEL

Tabel 1. Analisis Efisiensi dan Efektivitas Bot.....	9
Tabel 2. Variabel Robot.....	15
Tabel 3. Fungsi Prosedur Robot.....	16
Tabel 4. Fungsi Meriam Robot.....	16
Tabel 5. Fungsi Lifecycle.....	16

BAB 1

1. DESKRIPSI TUGAS

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

1.1. Spesifikasi Wajib

- Membuat 4 bot (1 utama dan 3 alternatif) dalam bahasa C# (.net) yang mengimplementasikan algoritma Greedy pada bot permainan Robocode Tank Royale.
- Strategi greedy yang diimplementasikan harus berbeda untuk setiap bot dan menggunakan heuristic yang berbeda.
- Bot yang dibuat tidak boleh sama dengan sampel yang diberikan sebagai contoh.
- Program harus mengandung komentar yang jelas.

1.2. Komponen Skor

- Bullet Damage: **Poin sebesar damage** yang dibuat kepada musuh menggunakan peluru.
- Bullet Damage Bonus: **20% dari damage** saat peluru membunuh musuh.
- Survival Score: **50 poin** setiap ada bot yang mati.
- Last Survival Bonus: **10 poin dikali banyaknya musuh bagi bot terakhir yang bertahan.**
- Ram Damage: **2x damage** yang dibuat kepada musuh dengan cara menabrak.
- Ram Damage Bonus: **30% dari damage** saat musuh terbunuh dengan ditabrak.

BAB 2

2. LANDASAN TEORI

2.1. Algoritma Greedy

Algoritma greedy adalah suatu pendekatan algoritma yang membuat pilihan yang tampak terbaik pada setiap langkah dengan harapan bahwa pilihan lokal yang optimal akan mengarah pada solusi global yang optimal. Algoritma greedy bekerja dengan cara memilih opsi terbaik yang tersedia pada saat itu tanpa mempertimbangkan konsekuensi jangka panjang.

Elemen-elemen utama dalam algoritma greedy meliputi:

- Himpunan Kandidat: Sekumpulan pilihan yang mungkin dapat diambil pada setiap langkah.
- Himpunan Solusi: Kumpulan kandidat yang telah dipilih untuk membentuk solusi.
- Fungsi Seleksi: Fungsi yang memilih kandidat terbaik dari himpunan kandidat.
- Fungsi Kelayakan: Fungsi yang memeriksa apakah kandidat yang dipilih layak untuk dimasukkan ke dalam solusi.
- Fungsi Objektif: Fungsi yang mengukur nilai solusi yang sedang dibangun.

2.2. Cara Kerja Program

Bot Robocode Tank Royale bekerja dengan cara merespons berbagai event yang terjadi selama pertempuran. Setiap turn, bot menerima informasi tentang posisinya, orientasinya, dan bot musuh yang terdeteksi oleh radar. Secara umum, aksi yang dilakukan adalah sebagai berikut :

- Pemindaian (Radar)
Radar merupakan komponen utama yang digunakan bot untuk mendeteksi keberadaan musuh di arena. Radar dapat berputar hingga 45 derajat per turn secara independen atau bersamaan dari body dan gun. Bot hanya dapat mendeteksi musuh yang berada dalam sudut pemindaian (scan arc), yaitu area yang disapu oleh radar dari posisi sebelumnya ke posisi saat ini dalam satu turn. Jika radar tidak bergerak, scan arc menjadi nol derajat sehingga tidak ada musuh yang terdeteksi. Oleh karena itu, radar harus selalu diputar setiap turn. Ketika musuh terdeteksi, event **OnScannedBot** akan dipanggil dan bot menerima informasi posisi, arah, kecepatan, dan energi musuh tersebut.
- Penembakan (Gun)
Gun dapat berputar hingga 20 derajat per turn secara independen atau bersamaan dengan body. Untuk menembak, perintah **Fire(firePower)** dipanggil dengan nilai firepower antara 0.1 hingga 3.0. Semakin besar firepower, semakin berat peluru dan semakin besar damage yang dihasilkan, namun peluru bergerak lebih lambat. Setelah menembak, gun akan menjadi panas dan tidak bisa menembak lagi sampai suhunya turun ke nol. Bot juga mendapatkan energi kembali sebesar 3x firepower jika peluru mengenai musuh.
- Pergerakan (Body)
Body tank dapat bergerak maju dan mundur dengan kecepatan maksimum 8 unit per turn, dengan percepatan 1 unit per turn dan perlambatan 2 unit per turn. Body juga dapat berbelok hingga 10 derajat per turn, namun kecepatan belok berkurang seiring bertambahnya kecepatan gerak. Bot akan kehilangan energi jika menabrak dinding (wall damage) atau menabrak bot lain. Jika bot menabrak musuh dengan bergerak maju, hal ini disebut ramming dan memberikan skor tambahan berupa Ram Damage.
- Reaksi terhadap Event
Bot merespons berbagai event yang terjadi selama pertempuran melalui method handler. **OnScannedBot** dipanggil saat radar mendeteksi musuh. **OnHitByBullet** dipanggil saat bot terkena peluru musuh. **OnHitWall** dipanggil saat bot menabrak dinding arena. **OnHitBot** dipanggil saat bot

bertabrakan dengan bot lain. **OnBotDeath** dipanggil saat ada bot yang mati di arena. Setiap event memberikan informasi yang dapat digunakan bot untuk menentukan aksi selanjutnya.

Implementasi algoritma greedy pada bot dilakukan dengan cara:

- Pada setiap turn, bot mengevaluasi semua pilihan aksi yang tersedia (bergerak, menembak, memutar radar, dll).
- Bot memilih aksi yang memberikan keuntungan terbesar secara lokal berdasarkan heuristic yang didefinisikan.
- Saat bot menerima sebuah event (seperti musuh terdeteksi, terkena peluru, atau menabrak dinding), bot langsung mengevaluasi kondisi tersebut dan mengeksekusi aksi terbaik secara lokal pada turn yang sama untuk memaksimalkan skor.

Cara Menjalankan Bot:

- Windows

Di bawah ini kasus sample bot bawaan github

1. **Build Game Engine**

Buka Powershell, lalu jalankan :

```
cd tank-royale-0.30.0
./gradlew :gui-app:clean
./gradlew :gui-app:build
```

2. **Jalankan Game Engine**

Buka PowerShell baru, lalu jalankan :

```
cd tank-royale-0.30.0
java -jar ./gui-app/build/libs/robocode-tankroyale-gui-0.30.0.jar
```

3. **Build Bot**

Buka PowerShell baru, masuk ke folder bot, lalu jalankan :

```
cd sample-bots/csharp>NamaBot
dotnet clean
dotnet restore
dotnet build
```

4. **Jalankan Bot**

Masih di terminal yang sama, jalankan:

```
$env:SERVER_SECRET = "isi_dengan_secret_dari_server.properties"
dotnet run
```

5. **Mulai Pertandingan di GUI**

1. Di GUI Robocode, klik **Config** → **Bot Root Directories**

2. Klik **Add** dan arahkan ke folder **sample-bots/csharp**

3. Klik **OK**

4. Klik **Battle** → **Start Battle** Di bagian **Joined Bots**, pilih bot yang sudah terhubung

5. Klik **Add All** lalu **Start Battle**

- Linux

Di bawah ini kasus bukan sample bots bawaan github

1. **Build Game Engine**

Buka Vscode terminal ubuntu, lalu jalankan :

```
cd tank-royale-0.30.0
./gradlew :gui-app:clean
./gradlew :gui-app:build
```

2. **Jalankan Game Engine**

Buka terminal baru, lalu jalankan :

```
cd tank-royale-0.30.0
java -jar ./gui-app/build/libs/robocode-tankroyale-gui-0.30.0.jar
```

Lalu mulai local server

3. **Build Bot**

Buka terminal baru, masuk ke folder bot, jalankan:

```
dotnet clean
dotnet restore
dotnet build
```

4. **Jalankan Bot**

Masih di terminal yang sama, jalankan:

```
export SERVER_SECRET=isi_dengan_secret_dari_server.properties
chmod +x NamaFolderBot.sh
./NamaFolderBot.sh
```

5. **Mulai Pertandingan di GUI**

Bila saat start local server hasilnya tidak merah, melainkan success hijau, maka:

1. Karena file sh sudah di jalankan, cek di Start Battle pada Game Engine
2. Bila berhasil, maka ada nama bot di kotak local/remote server di kiri bawah
3. Add bot bot tersebut
4. Lalu start battle untuk memulai pertempuran antar bot yang dipilih

BAB 3

3. APLIKASI STRATEGI GREEDY

3.1. Mapping Persoalan ke Elemen Greedy

Berikut adalah pemetaan persoalan Robocode Tank Royale ke dalam elemen-elemen algoritma greedy:

Himpunan Kandidat

Semua aksi yang dapat dilakukan bot pada setiap turn, yaitu: bergerak maju/mundur, berbelok, memutar gun, memutar radar, menembak dengan berbagai firepower, dan diam.

Himpunan Solusi

Kombinasi aksi yang dipilih bot pada setiap turn untuk memaksimalkan total skor akhir pertempuran.

Fungsi Seleksi

Fungsi yang memilih aksi terbaik berdasarkan heuristic yang digunakan, misalnya memilih target terdekat, memilih firepower berdasarkan jarak, atau memilih arah gerakan yang menghindari peluru.

Fungsi Kelayakan

Memeriksa apakah aksi yang dipilih layak dilakukan, misalnya: gun tidak sedang panas (untuk menembak), bot tidak di tepi arena (untuk bergerak maju), energi cukup (untuk menembak dengan firepower tinggi).

Fungsi Objektif

Memaksimalkan total skor yang terdiri dari: Bullet Damage, Bullet Damage Bonus, Survival Score, Last Survival Bonus, Ram Damage, dan Ram Damage Bonus.

3.2. Eksplorasi Alternatif Solusi Greedy

Bot 1: BotSweeper (Main Bot)

Sifat Greedy: Greedy pada **coverage** - selalu bergerak maju sejauh mungkin sambil memutar gun 360 derajat untuk memaksimalkan area yang dipindai dan ditembak, setiap scan akan menembak seketika.

Heuristic: Maksimalkan Bullet Damage dengan DPS (Damage Per Second) konstan menggunakan tembakan firepower 1.5 setiap kali musuh terdeteksi tanpa kalkulasi apapun, sambil terus berpatroli melingkar.

Bot 2: BotPredator

Sifat Greedy: Greedy pada **target locking** — bot langsung mengunci target pertama yang ditemukan dan terus mengejarnya sampai mati, tanpa mempertimbangkan apakah ada target lain yang lebih menguntungkan (lebih dekat atau lebih lemah).

Heuristik: Maksimalkan Ram Damage + Bullet Damage dengan strategi **pursue-and-ram** yaitu:

- Selalu maju ke arah target terkunci sejauh 10000 piksel (gas penuh)
- Menembak firepower 1.5 saat mengejar untuk cicil damage
- Saat berhasil menabrak, mundur 1 piksel lalu tembak firepower maksimal 3.0 untuk eksekusi
- Radar sweep 90 derajat di depan agar fokus pada target yang sedang dikejar, bukan menyapu arena secara luas

Bot 3: BotSentinel

Sifat Greedy: Greedy pada **posisi** - langsung mencari dan menempel di tembok terdekat sebagai posisi bertahan optimal, lalu menyapu area terbuka dengan gun secara terus-menerus.

Heuristic: Maksimalkan Survival Score dengan meminimalkan area yang bisa diserang (menempel tembok) sambil memaksimalkan scan coverage dengan rotasi gun 360 derajat. Saat kontak langsung dengan musuh, tembak firepower maksimal (3.0).

Bot 4: SniperBot (Alternatif 3)

Strategi Greedy : Bersembunyi di sudut arena dan menembak dengan peluru berat dari jauh.

Heuristic : Maksimalkan Bullet Damage Bonus dengan selalu menggunakan firepower maksimal (3.0) dan predictive aiming untuk akurasi tinggi, sambil meminimalkan risiko terkena serangan balik.

3.3. Analisis Efisiensi dan Efektivitas

Tabel 1. Analisis Efisiensi dan Efektivitas Bot

Nama Bot	BotPredator	BotSweeper	BotSentinel	BotSniper
Kompleksitas	O(1) per turn	O(1) per turn	O(1) per turn	O(1) per turn
Strategi Greedy	Mengunci satu target lalu mengejar dan menabrak	Menyapu arena dan menembak setiap musuh yang terlihat	Mengambil posisi tembok untuk bertahan	Menjaga jarak ideal dan menembak hanya saat akurat
Kelebihan	Damage besar, kuat 1v1, bisa mendapat skor tinggi jika berhasil ram	Konsisten, sederhana, coverage luas, tidak bergantung pada satu target	Survival tinggi, sulit diserang dari semua arah	Lebih adaptif, hemat energi, akurasi lebih baik
Kelemahan	Rentan ditembak banyak musuh karena gerakan agresif dan mudah terbaca	Akurasi tidak sepresisi sniper, tidak punya target selection	Pasif, damage bisa rendah jika musuh tidak mendekat	Bisa terlalu pasif dan kehilangan peluang damage
Efektivitas	Bagus untuk duel, kurang stabil untuk banyak musuh	Paling efektif berdasarkan 5 match karena konsisten peringkat 1	Cocok untuk bertahan, tapi kurang agresif	Baik untuk stabilitas, tapi kurang eksplosif

Berdasarkan hasil uji 5 match, BotSweeper dipilih sebagai bot utama karena paling konsisten memperoleh peringkat 1. Secara teori, BotSweeper memang bukan bot paling kompleks, tetapi justru kesederhanaannya menjadi keunggulan. Bot ini tidak bergantung pada satu target, tidak terlalu banyak kondisi, dan terus menghasilkan tekanan melalui gerakan patrol serta sweep gun 360°.

Dengan pendekatan greedy sederhana, yaitu bergerak terus, menyapu seluruh arena, dan menembak setiap musuh yang terdeteksi, BotSweeper mampu menjaga damage output secara stabil. Dibanding Predator yang agresif tetapi berisiko, Sentinel yang defensif, dan Sniper yang selektif, Sweeper lebih cocok sebagai solusi utama karena hasil eksperimen menunjukkan performanya paling konsisten dalam kondisi match sebenarnya.

3.4. Strategi Greedy yang Dipilih

Strategi greedy yang dipilih sebagai bot utama adalah BotSweeper dengan alasan:

Performa paling konsisten berdasarkan hasil eksperimen

- Dari 5 match yang dilakukan, BotSweeper paling sering memperoleh peringkat 1.
- Konsistensi dianggap lebih penting dibanding kemenangan sesaat karena menunjukkan strategi bekerja stabil di berbagai kondisi arena.

Coverage area sangat luas

- Gun diputar 360 derajat secara terus-menerus sehingga hampir seluruh area arena selalu terkena sapuan radar dan peluru.
- Hal ini meningkatkan peluang mengenai banyak musuh tanpa perlu memilih target spesifik.

Strategi greedy sederhana tetapi efektif

- Bot langsung menembak setiap musuh yang terdeteksi tanpa perhitungan kompleks.
- Keputusan lokal “lihat musuh → langsung tembak” membuat respons bot sangat cepat dan stabil.

Tidak bergantung pada satu target

- Berbeda dengan BotPredator yang hanya fokus mengejar satu musuh, BotSweeper dapat memberikan tekanan ke banyak musuh sekaligus.
- Dalam arena ramai, pendekatan ini lebih efektif karena selalu ada peluang damage ke berbagai arah.

Gerakan patroli membuat bot terus aktif

- Pola gerak melingkar dan maju terus membuat bot tidak diam di satu titik.
- Hal ini mengurangi risiko menjadi sasaran empuk bagi bot lain.

Implementasi ringan dan kompleksitas rendah

- Semua aksi dilakukan dalam kompleksitas $O(1)$ per turn.
- Tidak ada kalkulasi berat seperti prediksi posisi, tracking target kompleks, atau evaluasi multi-target.

Minim risiko bug logika

- Karena algoritmanya sederhana, kemungkinan error logika lebih kecil dibanding bot yang terlalu kompleks.
- Hal ini membuat perilaku bot lebih stabil selama pertandingan berlangsung.

Efektif pada arena dengan banyak musuh

- Semakin banyak musuh, semakin besar peluang sweep gun mengenai target.
- Strategi greedy coverage area menjadi lebih optimal pada mode free-for-all dibanding duel 1v1.

Damage output stabil

- Walaupun damage per tembakan tidak sebesar sniper atau predator, frekuensi tembakan BotSweeper sangat tinggi.
- Akumulasi damage yang konsisten menghasilkan performa yang baik secara keseluruhan.

Kemampuan recovery cepat saat menabrak

- Saat terkena tembok atau bertabrakan dengan bot lain, BotSweeper langsung membalik arah.
- Strategi ini mencegah bot terjebak (stuck) terlalu lama dan menjaga mobilitas tetap aktif.

Tidak terlalu bergantung pada akurasi tinggi

- BotSniper membutuhkan aim presisi sebelum menembak.
- BotSweeper tidak membutuhkan syarat tersebut sehingga lebih sering menghasilkan tekanan peluru di arena.

Lebih adaptif terhadap chaos arena

- Pada kondisi banyak bot bergerak bersamaan, strategi sederhana dan agresif area-wide lebih efektif dibanding strategi target tunggal.
- Bot tetap bisa menghasilkan damage walaupun posisi musuh terus berubah.

BAB 4

4. IMPLEMENTASI DAN PENGUJIAN

4.1. implementasi (Pseudocode)

4.1.1. BotPredator

PROCEDURE Run:

- Set warna bot
- Pisahkan radar dan gun dari gerakan badan

WHILE bot masih hidup:
 turnsSinceLastScan++

 IF tidak scan musuh selama 10 turn:
 targetBotId = -1
 Putar radar 360 derajat
 Reset timer pencarian

 ELSE IF target sudah ada:
 Arahkan badan ke target
 Arahkan gun ke target
 Maju sangat jauh
 Tembak power 1.5
 Sweep radar depan
 Eksekusi aksi

 ELSE:
 Sweep radar depan
 Eksekusi aksi

PROCEDURE SweepRadarFront:

 IF radarSweepRight:
 Radar sweep ke kanan/kiri area depan
 ELSE:
 Radar sweep ke arah sebaliknya
 Balik arah sweep radar

PROCEDURE OnScannedBot(e):

 Reset turnsSinceLastScan
 IF belum ada target OR musuh yang discan adalah target:
 Simpan ID target
 Simpan posisi target X dan Y

PROCEDURE OnHitBot(e):

 Mundur sedikit
 Arahkan gun ke target
 Tembak power 3.0
 Maju lagi

PROCEDURE OnHitWall(e):

 Mundur sedikit
 IF ada target:
 Belok ke arah target
 ELSE:
 Belok 90 derajat

PROCEDURE OnBotDeath(e):

```
IF bot yang mati adalah target:  
  Lepas target  
  Paksa scan ulang
```

4.1.2. BotSweeper

```
PROCEDURE Run:  
  Set warna bot  
  movingForward = true  
  
  WHILE bot masih hidup:  
    SetForward(40000)  
    movingForward = true  
  
    TurnRight(45)  
  
    TurnGunRight(360)  
  
PROCEDURE OnScannedBot(e):  
  Tembak langsung power 1.5  
  
PROCEDURE OnHitWall(e):  
  Panggil ReverseDirection()  
  
PROCEDURE OnHitBot(e):  
  IF bot tertabrak / terjadi ram:  
    Panggil ReverseDirection()  
  
PROCEDURE ReverseDirection:  
  IF movingForward == true:  
    SetBack(40000)  
    movingForward = false  
  ELSE:  
    SetForward(40000)  
    movingForward = true
```

4.1.3. BotSentinel

```
PROCEDURE Run:  
  Set warna bot  
  Pisahkan gun dari badan  
  
  TurnRight(Direction MOD 90)  
  Forward(50000)  
  
  sudahDiTembok = true  
  
  Back(15)  
  TurnRight(90)  
  
  WHILE bot masih hidup:  
    SetForward(40000 * arahGerak)  
    TurnGunRight(360)  
  
PROCEDURE OnScannedBot(e):  
  Tembak power 1.5  
  
PROCEDURE OnHitBot(e):  
  Tembak power 3.0  
  
PROCEDURE OnHitWall(e):
```

```
IF sudahDiTembok == true:
    IF arahGerak == 1:
        Back(15)
    ELSE:
        Forward(15)

    arahGerak = arahGerak * -1
```

4.1.4. BotSniper

PROCEDURE Run:

```
Set warna bot
Pisahkan radar dan gun dari badan

WHILE bot masih hidup:
    IF waktuLepasTembok > 0:
        waktuLepasTembok--

    waktuHilang++

    IF target hilang lebih dari 8 turn:
        Putar radar 360 derajat
        Belok sedikit sesuai arahStrafe
        Maju pelan

    Eksekusi aksi
```

PROCEDURE OnScannedBot(e):

```
waktuHilang = 0

Hitung arah mutlak musuh
Hitung sudut ke musuh
Hitung jarak ke musuh

Lock radar ke musuh
Lock gun ke musuh

Panggil AturGerakanSniper(sudutKeMusuh, jarak)
Panggil TembakKalauAman(jarak, gunTurn)
```

PROCEDURE AturGerakanSniper(sudutKeMusuh, jarak):

```
IF sedang lepas dari tembok:
    Belok tajam
    Maju keluar dari sudut map
    RETURN

IF jarak < JarakBahaya:
    Menjauh secara diagonal
    Maju cepat

ELSE IF jarak < JarakAman:
    Mundur/bergerak menyamping
    Maju sedang

ELSE IF jarak terlalu jauh:
    Mendekat sedikit secara diagonal

ELSE:
    Strafe melingkar di zona ideal
```

PROCEDURE TembakKalauAman(jarak, gunTurn):

IF jarak > JarakTembakMaks:
RETURN

IF sudut gun belum presisi:
RETURN

IF jarak dekat:
power = 3.0
ELSE IF jarak ideal:
power = 2.2
ELSE:
power = 1.4

Tembak dengan power tersebut

PROCEDURE OnHitBot(e):

Balik arah strafe
Belok menjauh dari musuh
Maju
Tembak power 3.0

PROCEDURE OnHitWall(e):

Balik arah strafe
Aktifkan waktuLepasTembok

4.2. Struktur Data, Fungsi, dan Prosedur

Variabel

Tabel 2. Variabel Robot

Nama	Asal	Tipe	Penjelasan
movingForward	Custom	bool	Menyimpan arah gerak tank saat ini. true = sedang maju, false = sedang mundur. Dipakai oleh ReverseDirection() agar tahu harus set apa.
IsRunning	Robocode (Bot.cs)	bool (read-only)	Bernilai true selama ronde berlangsung. Begitu ronde selesai atau bot mati, nilainya jadi false dan loop while berhenti otomatis.
e (parameter event)	Robocode	object event	Objek yang dikirim engine saat event terjadi, berisi data situasi saat itu (posisi musuh, siapa yang menabrak, dll). Nama e hanya konvensi penulisan.

Fungsi/Prosedur Bawaan Robocode

Semua ini didapat dari warisan kelas Bot.cs dan BaseBot.cs

Pergerakan Badan

Tabel 3. Fungsi Prosedur Robot

Nama	Penjelasan
SetForward(double distance)	Non-blocking. Mendaftarkan perintah maju sejauh distance unit. Perintah baru dieksekusi saat Go() dipanggil (atau di akhir tick). Tidak menunggu bot benar-benar sampai.
SetBack(double distance)	Non-blocking. Sama seperti SetForward tapi mundur. Internally memanggil SetForward(-distance).
TurnRight(double degrees)	Blocking. Memutar badan bot ke kanan sejumlah derajat dan menunggu sampai putaran selesai sebelum lanjut ke baris berikutnya.

Meriam (Gun)

Tabel 4. Fungsi Meriam Robot

Nama	Penjelasan
TurnGunRight(double degrees)	Blocking. Memutar laras meriam ke kanan dan menunggu sampai selesai. Dipanggil TurnGunRight(360) agar laras menyapu seluruh sudut arena.
Fire(double firepower)	Blocking. Menembakkan peluru dengan daya tembak antara 0.1 sampai 3.0. Semakin besar firepower, makin besar damage tapi makin lambat peluru dan makin lama gun cooldown. Di sini dipakai 1.5 sebagai nilai tengah.

Lifecycle & Loop

Tabel 5. Fungsi Lifecycle

Nama	Penjelasan
Run()	Override. Metode utama bot. Dipanggil otomatis oleh engine di awal setiap ronde. Berisi loop while (IsRunning) sebagai jantung logika bot.
Start()	Dipanggil di Main() untuk menghubungkan bot ke server Robocode dan memulai sesi. Tanpa ini bot tidak jalan sama sekali.
BotInfo.FromFile("BotSweeper.json")	Membaca file konfigurasi JSON (nama, author, deskripsi bot) dan menjadikannya objek BotInfo yang dikirim ke konstruktor.

Event Handler (Override dari Robocode)

Semua dipanggil otomatis oleh engine — kita hanya mengisi isinya.

Nama	Kapan Dipanggil	Data dalam e
OnScannedBot(ScannedBotEvent e)	Setiap kali radar mendeteksi bot musuh.	Posisi musuh (e.X, e.Y), ID musuh (e.ScannedBotId), energi, kecepatan, dll.
OnHitWall(HitWallEvent e)	Saat badan bot menyentuh dinding arena.	Arah benturan (e.Bearing).
OnHitBot(HitBotEvent e)	Saat badan bot bersentuhan/bertabrakan dengan bot lain.	e.IsRammed (bool), e.BotId, energi lawan.

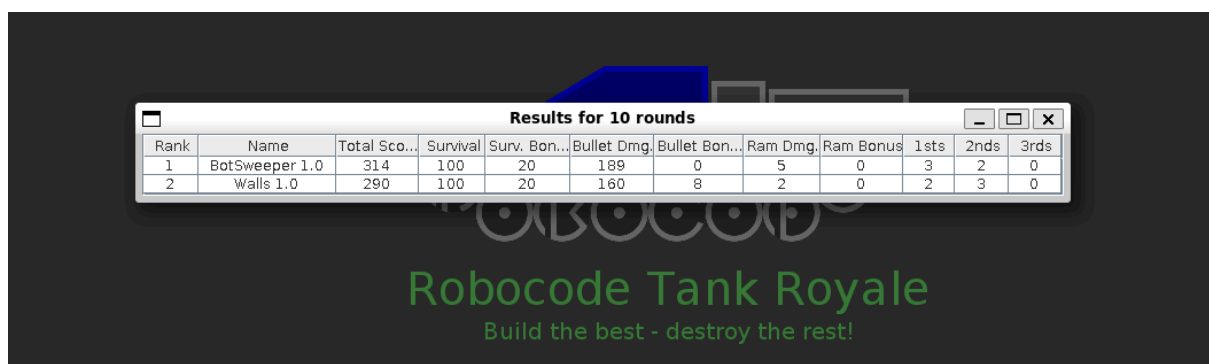
Properti penting dalam event OnHitBot

Nama	Penjelasan
e.IsRammed	true jika bot lain yang aktif menabrak kita (mereka bergerak ke arah kita). false jika kita yang menabrak mereka. Di BotSweeper, jika IsRammed maka kita yang tertabrak → balik arah.

Fungsi Custom (Buatan Sendiri)

Nama	Penjelasan
ReverseDirection()	Prosedur helper yang membalik arah gerak. Cek nilai movingForward: kalau true → panggil SetBack(40000) dan set false; kalau false → panggil SetForward(40000) dan set true. Dipanggil dari OnHitWall dan OnHitBot.

4.3. Pengujian



Pengujian dilakukan sebanyak 10 kali(round) dengan format 1 vs 1, antara **BotSweeper** dengan BotAsisten (BotSample Walls).

Kejadian unik tidak ditemukan dalam pengujian ini. pengujian ini kami tidak mendapat hasil optimal dikarenakan kekhususan bot lawan dalam menghindari, bot sample lawan yang digunakan menggunakan algoritma yang menempel tembok dan jalan di sana mengelilingi tembok, hal ini membuat BotSweeper kesulitan mengenainya saat menembakkan peluru. Pada saat BotSweeper mengscan Bot Walls, peluru tidak banyak tidak sampai ke tubuh Bot Walls dikarenakan kecenderungannya untuk selalu bergerak di tembok (sangat jauh). Dapat disimpulkan bahwa BotSweeper tidak berhasil mencapai nilai optimal ketika musuh sedikit dan musuh tersebut suka menjauh dan bergerak cepat.

4.4. Analisis Hasil Pengujian

Hasil dari Analisis pengujian ini kami tidak mendapat hasil optimal dikarenakan kekhususan bot lawan dalam menghindari, bot sample lawan yang digunakan menggunakan algoritma yang menempel tembok dan jalan di sana mengelilingi tembok, hal ini membuat BotSweeper kesulitan mengenainya saat menembakkan peluru. Pada saat BotSweeper mengscan Bot Walls, peluru tidak banyak tidak sampai ke tubuh Bot Walls dikarenakan kecenderungannya untuk selalu bergerak di tembok (sangat jauh). Dapat disimpulkan bahwa BotSweeper tidak berhasil mencapai nilai optimal ketika musuh sedikit dan musuh tersebut suka menjauh dan bergerak cepat.

BAB 5

5. KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, algoritma greedy berhasil diterapkan pada BotSweeper melalui strategi sederhana, yaitu bergerak aktif, melakukan sweep meriam 360 derajat, dan langsung menembak setiap musuh yang terdeteksi. Strategi ini terbukti cukup efektif dalam beberapa skenario pertandingan karena BotSweeper mampu memberikan tekanan secara konsisten tanpa harus memilih target secara spesifik.

Namun, hasil pengujian melawan BotWalls menunjukkan bahwa performa BotSweeper belum mencapai hasil yang optimal. Perbedaan hasil antara BotSweeper sebagai bot utama dan BotWalls tidak terlalu signifikan karena BotWalls memiliki kemampuan bertahan yang sangat baik. BotWalls menggunakan strategi menempel dan bergerak di sepanjang tembok, sehingga posisinya cenderung jauh dan sulit dijangkau oleh tembakan BotSweeper.

Dari hasil analisis, BotSweeper mengalami kesulitan ketika jumlah musuh sedikit dan musuh tersebut memiliki pola gerak defensif, cepat, serta cenderung menjauh. Saat BotSweeper berhasil melakukan scan terhadap BotWalls, peluru yang ditembakkan tidak selalu mengenai target karena BotWalls terus bergerak di area tembok. Dengan demikian, dapat disimpulkan bahwa strategi greedy BotSweeper efektif untuk kondisi arena ramai, tetapi kurang optimal saat menghadapi bot defensif yang menjaga jarak dan bergerak cepat di sepanjang tembok.

5.2. Saran

Untuk pengembangan selanjutnya, BotSweeper dapat ditingkatkan dengan menambahkan strategi yang lebih adaptif, seperti sistem prediksi posisi musuh, penguncian target, dan pengaturan kekuatan tembakan berdasarkan jarak. Selain itu, bot juga dapat diberi kemampuan untuk mengenali pola gerakan lawan, terutama lawan yang bergerak di sepanjang tembok.

Strategi greedy yang digunakan BotSweeper masih dapat dipertahankan, tetapi perlu dikombinasikan dengan heuristik tambahan agar bot tidak hanya menembak saat musuh terdeteksi, melainkan juga memperkirakan posisi musuh berikutnya. Dengan begitu, BotSweeper dapat menjadi lebih efektif dalam menghadapi bot defensif seperti BotWalls maupun bot yang memiliki pola gerak cepat dan sulit ditebak.

LAMPIRAN

Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.	✓	

Tautan Repository GitHub

Repository: https://github.com/FeldyAja/Tubes_BotAbis

Tautan Video (Nilai Bonus 80%)

<https://www.youtube.com/live/qByJSp7lHsA?si=3cpPYdmVmUZ8AquU>

DAFTAR PUSTAKA

- [1] Robocode Dev Team, “Robocode Tank Royale,” GitHub Repository.
<https://github.com/robocode-dev/tank-royale>.
- [2] Robocode Dev Team, “Robocode Tank Royale Documentation,”
<https://robocode-dev.github.io/tank-royale/>.
- [3] Microsoft, “.NET,”
<https://dotnet.microsoft.com/en-us/>.
- [4] Microsoft, “Learn to Code C#,”
<https://dotnet.microsoft.com/en-us/learn/csharp>.
- [5] Microsoft, “C# Language Documentation,”
<https://learn.microsoft.com/en-us/dotnet/csharp/>.
- [6] Amazon Web Services, “What is an API?,”
<https://aws.amazon.com/what-is/api/>.